

CSCI-B 651 NLP Spring 2026 Homework 3

April 5, 2026

Instructions

We will be using Canvas to collect your assignments. Please read the following instructions to prepare your submission.

1. Submit your solution in a pdf file and a zip file (<yourFirstName_LastName>.pdf/zip). Your write-up must be in pdf. Your code must be in the zip file.
2. In your pdf file, the solution to each problem should start on a new page.
3. Latex is strongly encouraged to write your solutions, e.g., using Overleaf (<https://www.overleaf.com/>). However, scanned handwritten copies are also acceptable. Hard copies will not be accepted.
4. You need to add screen captures of your code in your write-up.
5. You may discuss the problems and potential directions for solving them with another student. However, you need to write your own solutions and code separately, and not as a group activity. Please list the students you collaborated with on your submission.

Problem 1 (Programming involved, 30 points)

In this assignment, we will build a classifier using a pre-trained language model. We will use python packages, Hugging Face, PyTorch, Scikit-Learn, and NumPy. Please use up-to-date versions of the packages for this assignment.

Data

We will use the *polarity dataset v2.0* which was used for Homework 2. You can download the dataset from this website. When uncompressing the file, you will find data in two folders: *pos* and *neg*.

We will prepare our training and test datasets as in Homework 2, but with 9:1. Sort the files in each folder in ascending order using the filenames. Move the last 100 samples in each folder to new folders (*pos_test*, *neg_test*), which will be our test data. Change the original folder names into *pos_train* and *neg_train*, respectively. Now your *pos_train* and *neg_train* have 900 samples each, and *pos_test*, *neg_test* have 100 samples each. The files in *_train* and *_test* folders must be different.

pos and *neg* will be the labels for the documents. When you read the documents in your code, your code will have to remember the label for each document.

Coding Instructions

We will refer to this Hugging Face tutorial to build your own classifier. Write code following the steps below, referring to the tutorial. Include screenshots of your code in your write-up for each step.

- (a) Implement a function called `read_data(...)`. The input of this function will be paths to *pos_train*, *neg_train*, *pos_test*, and *neg_test*. The output (return values) will be *train_texts*, *train_labels*, *test_texts*, and *test_labels*. We will tokenize the texts using the *DistilBert* tokenizer in the following steps.

```
# a
import os

def read_data(pos_train_path, neg_train_path, pos_test_path, neg_test_path):
    def load_folder(folder_path, label):
        texts = []
        labels = []
        # Sort filenames in ascending order to ensure consistent ordering
        filenames = sorted(os.listdir(folder_path))
        for fname in filenames:
            if fname.endswith('.txt'):
                fpath = os.path.join(folder_path, fname)
                # Use errors='replace' to handle any unusual encodings in the dataset
                with open(fpath, 'r', encoding='utf-8', errors='replace') as f:
                    texts.append(f.read())
                labels.append(label)
        return texts, labels

    # Load positive and negative training data
    pos_train_texts, pos_train_labels = load_folder(pos_train_path, label=1)
    neg_train_texts, neg_train_labels = load_folder(neg_train_path, label=0)

    # Load positive and negative test data
    pos_test_texts, pos_test_labels = load_folder(pos_test_path, label=1)
    neg_test_texts, neg_test_labels = load_folder(neg_test_path, label=0)

    # Merge positive and negative samples for train and test
    train_texts = pos_train_texts + neg_train_texts
    train_labels = pos_train_labels + neg_train_labels

    test_texts = pos_test_texts + neg_test_texts
    test_labels = pos_test_labels + neg_test_labels

    return train_texts, train_labels, test_texts, test_labels

train_texts, train_labels, test_texts, test_labels = read_data(
    pos_train_path='pos_train',
    neg_train_path='neg_train',
    pos_test_path='pos_test',
    neg_test_path='neg_test'
)
```

- (b) We will create a development dataset. Using the `train_test_split` function in *sklearn*, split the training data into training data (`train_texts` and `train_labels`) and development data (`dev_texts` and `dev_labels`). Make the dev dataset contain 100 positive samples and 100 negative samples. You can do this by changing the parameters of the function.

```
# b
from sklearn.model_selection import train_test_split
train_texts, dev_texts, train_labels, dev_labels = train_test_split(
    train_texts,
    train_labels,
    test_size=200/1800,
    stratify=train_labels,
    random_state=42
)
✓ 0.8s
```

- (c) Write code to tokenize the texts in the three datasets, using the `DistilBertTokenizerFast`.

```
# c
from transformers import DistilBertTokenizerFast

# Load the DistilBert tokenizer
tokenizer = DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')

# Tokenize train, dev, and test texts
# truncation=True : truncate sequences longer than max_length (512 tokens)
# padding=True : pad shorter sequences to the same length within each batch
train_encodings = tokenizer(train_texts, truncation=True, padding=True)
dev_encodings = tokenizer(dev_texts, truncation=True, padding=True)
test_encodings = tokenizer(test_texts, truncation=True, padding=True)
✓ 3.8s
```

- (d) Now convert the tokenized data and labels into a Dataset object. Declare a class `PolarityDataset`, and create `PolarityDataset` objects for each dataset (i.e., training, dev, and test)

```
# d
import torch

class PolarityDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        # Store the tokenized encodings and labels
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        # Retrieve a single sample by index as a dictionary of tensors
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        # Add the label for this sample
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        # Return the total number of samples in the dataset
        return len(self.labels)

# Create PolarityDataset objects for train, dev, and test
train_dataset = PolarityDataset(train_encodings, train_labels)
dev_dataset = PolarityDataset(dev_encodings, dev_labels)
test_dataset = PolarityDataset(test_encodings, test_labels)
✓ 0.0s
```

- (e) Write code for fine-tuning with native PyTorch, using the DistilBert pre-trained model. You can refer to the tutorial. Explain each line of the code using comments.

```
# e
from transformers import DistilBertForSequenceClassification
from torch.utils.data import DataLoader
from torch.optim import AdamW

# Load the pre-trained DistilBert model for sequence classification
# num_labels=2 because we have two classes: pos and neg
model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=2
)

# Use GPU if available, otherwise use CPU
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
model.to(device)

# Create DataLoaders for train, dev, and test datasets
# batch_size=16 as required in part (h)
# shuffle=True for training to prevent the model from learning the order of samples
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
dev_loader = DataLoader(dev_dataset, batch_size=16, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# Initialize the AdamW optimizer with learning rate 5e-5 as required in part (h)
optimizer = AdamW(model.parameters(), lr=5e-5)

# Set the number of epochs
num_epochs = 3

# Training loop
for epoch in range(num_epochs):
    # --- Training phase ---
    model.train() # Set model to training mode
    total_train_loss = 0

    for batch in train_loader:
        # Move each tensor in the batch to the correct device
        batch = {k: v.to(device) for k, v in batch.items()}

        # Forward pass: compute model output and loss
        outputs = model(**batch)
        loss = outputs.loss

        # Accumulate training loss
        total_train_loss += loss.item()

        # Backward pass: compute gradients
        loss.backward()

        # Update model parameters
        optimizer.step()

        # Clear gradients for the next step
        optimizer.zero_grad()

    # --- Validation phase ---
    model.eval() # Set model to evaluation mode
    total_dev_loss = 0

    with torch.no_grad(): # Disable gradient computation for validation
        for batch in dev_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            total_dev_loss += outputs.loss.item()

    # Calculate average losses
    avg_train_loss = total_train_loss / len(train_loader)
    avg_dev_loss = total_dev_loss / len(dev_loader)
```

✓ 31m 43.9s

- (f) Add code to print the training loss and validation loss (i.e., loss on the dev set) every epoch.

```
# f
print(f"Epoch {epoch+1}/{num_epochs} | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_dev_loss:.4f}")
```

```
Epoch 1/3 | Train Loss: 0.6813 | Val Loss: 0.6182
Epoch 2/3 | Train Loss: 0.4900 | Val Loss: 0.4277
Epoch 3/3 | Train Loss: 0.2409 | Val Loss: 0.4394
```

- (g) Add code to obtaining the performance of the trained model on the test data. Add code to print classification accuracy, precision, recall, and f1 score. [precision, recall, and f1 score from the positive ('pos') side]

```
# g
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Set model to evaluation mode
model.eval()

all_preds = []
all_labels = []

with torch.no_grad(): # Disable gradient computation for inference
    for batch in test_loader:
        # Move batch to the correct device
        batch = {k: v.to(device) for k, v in batch.items()}

        # Forward pass: get model outputs
        outputs = model(**batch)

        # Get predicted class by taking the argmax of the logits
        preds = torch.argmax(outputs.logits, dim=1)

        # Collect predictions and true labels
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(batch['labels'].cpu().numpy())

# Calculate metrics (pos=1 is the positive class)
accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds, pos_label=1)
recall = recall_score(all_labels, all_preds, pos_label=1)
f1 = f1_score(all_labels, all_preds, pos_label=1)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")
✓ 11.7s

Accuracy : 0.8850
Precision : 0.9425
Recall : 0.8200
F1 Score : 0.8770
```

- (h) Run the code with AdamW optimizer, learning rate 5e-5, and batch size 16, and the number of epochs 3. What is the training loss and validation loss on epoch 3? What is the test performance when applying the trained model to the test data? Answer these questions in your write-up.

- Epoch 3 Train Loss: 0.2409
- Epoch 3 Val Loss: 0.4394
- Test Accuracy: 0.8850

- Precision: 0.9425
- Recall: 0.8200
- F1 Score: 0.8770

- (i) Explore different learning rate, batch size, and the number of epochs. State the resulting changes together with what you changed and how you changed. Discuss the resulting changes (in terms of how and why).

Setting	LR	Batch	Epochs	Acc	Precision	Recall	F1
Baseline	5e-5	16	3	0.8850	0.9425	0.8200	0.8770
Exp 1: Higher LR	1e-4	16	3	0.5000	0.5000	1.0000	0.6667
Exp 2: Smaller Batch	5e-5	8	3	0.8500	0.8241	0.8900	0.8558
Exp 3: More Epochs	5e-5	16	5	0.8350	0.7724	0.9500	0.8520

Experiment 1: Higher Learning Rate (1e-4)

I increased the learning rate from 5e-5 to 1e-4. The model failed to converge, predicting all samples as positive and achieving only 50% accuracy. A larger learning rate causes gradient updates that are too large, destabilizing the optimization process and preventing the model from learning meaningful features.

Experiment 2: Smaller Batch Size (8)

I reduced the batch size from 16 to 8. Accuracy dropped slightly (0.8500 vs 0.8850) while recall increased (0.8900 vs 0.8200). Smaller batch sizes introduce more noise into gradient updates, which can help the model generalize but also reduces precision as the model becomes more aggressive in predicting positive samples.

Experiment 3: More Epochs (5)

I increased the number of epochs from 3 to 5. The model showed clear overfitting: training loss continued to decrease while validation loss increased from 0.5223 to 0.8160, and test accuracy dropped to 0.8350. Training for too many epochs causes the model to memorize the training data rather than learning generalizable features.

- (j) Make one change in the network architecture (not the optimizer/hyperparameters). State what change you made. Discuss the performance change (in terms of how and why).

I froze all layers in the DistilBERT transformer block (embeddings and all 6 transformer layers), leaving only the classification head (`pre_classifier` and `classifier`) trainable.

Setting	Accuracy	Precision	Recall	F1
Baseline (full fine-tuning)	0.8850	0.9425	0.8200	0.8770
Layer Freezing	0.7450	0.7816	0.6800	0.7273

Freezing the transformer layers resulted in a noticeable performance drop. Since only the classification head is updated, the model has limited capacity to adapt to task-specific features. The consistently high training loss (0.63–0.69) confirms that full fine-tuning of all layers is necessary for optimal performance on this task.

Problem 2 (20 points)

Below is a table that represents the occurrences of words in documents.

	Doc1	Doc2	Doc3	Doc4
cup	0	87	78	3
summer	10	12	15	50
coffee	25	18	0	0
walk	3	6	32	20
vacation	20	23	18	15

Answer the following questions. Show your work.

- (a) Compute TF-IDF for *coffee* in each document. You will get four results ($TFIDF_{coffee,doc1}$, $TFIDF_{coffee,doc2}$, $TFIDF_{coffee,doc3}$, $TFIDF_{coffee,doc4}$)
- $$TFIDF_{w,d} = TF_{w,d} \cdot IDF_w$$
- $$TFIDF_{coffee,doc1} = \log_{10}(25+1) \cdot \log_{10}\frac{4}{2} = 1.4150 \times 0.3010 \approx 0.4259$$
- $$TFIDF_{coffee,doc2} = \log_{10}(18+1) \cdot \log_{10}\frac{4}{2} = 1.2788 \times 0.3010 \approx 0.3849$$
- $$TFIDF_{coffee,doc3} = \log_{10}(0+1) \cdot \log_{10}\frac{4}{2} = 0$$
- $$TFIDF_{coffee,doc4} = \log_{10}(0+1) \cdot \log_{10}\frac{4}{2} = 0$$
- (b) Compute TF-IDF for *vacation* in each document. You will get four results. ($TFIDF_{vacation,doc1}$, $TFIDF_{vacation,doc2}$, $TFIDF_{vacation,doc3}$, $TFIDF_{vacation,doc4}$)
- $$TFIDF_{vacation,doc1} = \log_{10}(20+1) \cdot \log_{10}\frac{4}{4} = 0$$
- $$TFIDF_{vacation,doc2} = \log_{10}(23+1) \cdot \log_{10}\frac{4}{4} = 0$$
- $$TFIDF_{vacation,doc3} = \log_{10}(18+1) \cdot \log_{10}\frac{4}{4} = 0$$
- $$TFIDF_{vacation,doc4} = \log_{10}(15+1) \cdot \log_{10}\frac{4}{4} = 0$$
- (c) Compare TF-IDF values from (a) and (b). Discuss the results in terms of their discriminating power.
- The word coffee has discriminating power ($IDF = 0.3010 > 0$), enabling it to distinguish Doc1 and Doc2 from the other documents.
- The word vacation has absolutely no discriminating power ($IDF=0$) because it appears in all documents, making its TF-IDF values strictly 0 everywhere.
- (d) Compute cosine similarity between pairs of documents using TF-IDF representations of the documents. Which documents are most similar? Which documents are least similar?

	IDF	$TFIDF_{doc1}$	$TFIDF_{doc2}$	$TFIDF_{doc3}$	$TFIDF_{doc4}$
cup	$\log_{10}\frac{4}{2}$	0	0.243	0.237	0.075
summer	$\log_{10}\frac{4}{2}$	0	0	0	0
coffee	$\log_{10}\frac{4}{2}$	0.426	0.385	0	0
walk	$\log_{10}\frac{4}{2}$	0	0	0	0
vacation	$\log_{10}\frac{4}{4}$	0	0	0	0

$$\|\vec{doc1}\| = \sqrt{0^2 + 0^2 + 0.426^2 + 0^2} \approx 0.426$$

$$\|\vec{doc2}\| = \sqrt{0.243^2 + 0^2 + 0.385^2 + 0^2} \approx 0.455$$

$$\|\vec{doc3}\| = \sqrt{0.237^2 + 0^2 + 0^2 + 0^2} \approx 0.237$$

$$\|\vec{doc4}\| = \sqrt{0.075^2 + 0^2 + 0^2 + 0^2} \approx 0.075$$

$$\cos(\vec{doc1}, \vec{doc2}) = \frac{0 \cdot 0.243 + 0 \cdot 0 + 0.426 \cdot 0.385 + 0 \cdot 0}{0.426 \cdot 0.455} = 0.846$$

$$\cos(\vec{doc1}, \vec{doc3}) = \frac{0 \cdot 0.237 + 0 \cdot 0 + 0.426 \cdot 0 + 0 \cdot 0}{0.426 \cdot 0.237} = 0$$

$$\cos(\vec{doc1}, \vec{doc4}) = \frac{0 \cdot 0.075 + 0 \cdot 0 + 0.426 \cdot 0 + 0 \cdot 0}{0.426 \cdot 0.075} = 0$$

$$\begin{aligned} \cos(\vec{doc2}, \vec{doc3}) &= \frac{0.243 \cdot 0.237 + 0 \cdot 0 + 0.385 \cdot 0 + 0 \cdot 0}{0.455 \cdot 0.237} = 0.534 \\ \cos(\vec{doc2}, \vec{doc4}) &= \frac{0.243 \cdot 0.075 + 0 \cdot 0 + 0.385 \cdot 0 + 0 \cdot 0}{0.455 \cdot 0.075} = 0.534 \\ \cos(\vec{doc3}, \vec{doc4}) &= \frac{0.237 \cdot 0.075 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0}{0.237 \cdot 0.075} = 1 \end{aligned}$$

Most similar: Documents 3 and 4 are the most similar, with the highest cosine similarity score of 1.

Least similar: The pairs (Document 1, 3) and (Document 1, 4) are the least similar, with the lowest score of 0.

Problem 3 (20 points)

Below is a table that represents the occurrences of words and their context words. We are going to compute Positive Pointwise Mutual Information (PPMI) following what we learned in class. Answer the following questions. Show your work.

	keyboard	webcam	sheep	legs	racing
whiteboard	1434	512	8	42	3
computer	5290	2351	19	9	42
mouse	6102	3425	352	4251	532
horse	1	53	531	2542	3513
vacation	14	53	34	5	9

- (a) First, fill out the counts of word and context words, $count(w)$ and $count(context)$.

	keyboard	webcam	sheep	legs	racing	$count(w)$
whiteboard	1434	512	8	42	3	1999
computer	5290	2351	19	9	42	7711
mouse	6102	3425	352	4251	532	14662
horse	1	53	531	2542	3513	6640
vacation	14	53	34	5	9	115
$count(context)$	12841	6394	944	6849	4099	31127

- (b) Compute $p(w, context)$, $p(w)$, and $p(context)$.

	$p(w, context)$					$p(w)$
	keyboard	webcam	sheep	legs	racing	$p(w)$
whiteboard	0.0461	0.0165	0.0003	0.0013	0.0001	0.0642
computer	0.1700	0.0755	0.0006	0.0003	0.0014	0.2477
mouse	0.1960	0.1100	0.0113	0.1366	0.0171	0.4710
horse	0.00003	0.0017	0.0171	0.0817	0.1129	0.2133
vacation	0.0005	0.0017	0.0011	0.0002	0.0003	0.0037
$p(context)$	0.4126	0.2054	0.0303	0.2200	0.1317	1

- (c) Compute PPMI scores, and fill out the table.

	keyboard	webcam	sheep	legs	racing
whiteboard	0.799	0.320	0	0	0
computer	0.735	0.572	0	0	0
mouse	0.012	0.183	0	0.399	0
horse	0	0	1.401	0.803	2.008
vacation	0	1.159	3.298	0	0

- (d) Compute cosine similarity between pairs of words (e.g., whiteboard-computer, whiteboard-mouse, horse-vacation) using the PPMI word representations. Which words are most similar? Which words are least similar?

$$\| \vec{whiteboard} \| = \sqrt{0.799^2 + 0.32^2} \approx 0.861$$

$$\| \vec{computer} \| = \sqrt{0.735^2 + 0.572^2} \approx 0.931$$

$$\| \vec{mouse} \| = \sqrt{0.012^2 + 0.183^2 + 0.399^2} \approx 0.439$$

$$\| \vec{horse} \| = \sqrt{1.401^2 + 0.803^2 + 2.008^2} \approx 2.577$$

$$\| \vec{vacation} \| = \sqrt{1.159^2 + 3.298^2} \approx 3.496$$

$$\begin{aligned}
\cos(\vec{whiteboard}, \vec{computer}) &= \frac{0.799 \cdot 0.735 + 0.32 \cdot 0.572 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0}{0.861 \cdot 0.931} = 0.961 \\
\cos(\vec{whiteboard}, \vec{mouse}) &= \frac{0.799 \cdot 0.012 + 0.32 \cdot 0.183 + 0 \cdot 0 + 0 \cdot 0.399 + 0 \cdot 0}{0.861 \cdot 0.439} = 0.181 \\
\cos(\vec{whiteboard}, \vec{horse}) &= \frac{0.799 \cdot 0 + 0.32 \cdot 0 + 0 \cdot 1.401 + 0 \cdot 0.803 + 0 \cdot 2.008}{0.861 \cdot 2.577} = 0 \\
\cos(\vec{whiteboard}, \vec{vacation}) &= \frac{0.799 \cdot 0 + 0.32 \cdot 1.159 + 0 \cdot 3.298 + 0 \cdot 0 + 0 \cdot 0}{0.861 \cdot 3.496} = 0.123 \\
\cos(\vec{computer}, \vec{mouse}) &= \frac{0.735 \cdot 0.012 + 0.572 \cdot 0.183 + 0 \cdot 0 + 0 \cdot 0.399 + 0 \cdot 0}{0.931 \cdot 0.439} = 0.278 \\
\cos(\vec{computer}, \vec{horse}) &= \frac{0.735 \cdot 0 + 0.572 \cdot 0 + 0 \cdot 1.401 + 0 \cdot 0.803 + 0 \cdot 2.008}{0.931 \cdot 2.577} = 0 \\
\cos(\vec{computer}, \vec{vacation}) &= \frac{0.735 \cdot 0 + 0.572 \cdot 1.159 + 0 \cdot 3.298 + 0 \cdot 0 + 0 \cdot 0}{0.931 \cdot 3.496} = 0.204 \\
\cos(\vec{mouse}, \vec{horse}) &= \frac{0.012 \cdot 0 + 0.183 \cdot 0 + 0 \cdot 1.401 + 0 \cdot 3.298 + 0 \cdot 2.008}{0.439 \cdot 2.577} = 0.283 \\
\cos(\vec{mouse}, \vec{vacation}) &= \frac{0.012 \cdot 0 + 0.183 \cdot 1.159 + 0 \cdot 3.298 + 0 \cdot 3.999 + 0 \cdot 0}{0.439 \cdot 3.496} = 0.138 \\
\cos(\vec{horse}, \vec{vacation}) &= \frac{0 \cdot 0 + 0 \cdot 1.159 + 1.401 \cdot 3.298 + 0 \cdot 803 \cdot 0 + 2.008 \cdot 0}{2.577 \cdot 3.496} = 0.513
\end{aligned}$$

Most similar: Whiteboard and computer are the most similar pair, achieving the highest cosine similarity score of 0.961.

Least similar: The pairs (whiteboard, horse) and (computer, horse) are the least similar, both resulting in a cosine similarity score of 0.

Problem 4 (Programming involved, 30 points)

In this assignment, you will answer the questions using WordNet and . You will use the NLTK package to access WordNet. A NLTK WordNet tutorial is available here: <https://www.nltk.org/howto/wordnet.html>

- (a) Write the code to find hypernyms of the first definition of the word *house*. Print out the hypernyms.

```
HW > HW3 > run_wordnet.py > ...
1  import nltk
2  nltk.download('wordnet')
3
4  from nltk.corpus import wordnet as wn
5
6  # Get the first synset of "house"
7  house_synset = wn.synsets('house')[0]
8  print(f"First synset of 'house': {house_synset}")
9  print(f"Definition: {house_synset.definition()}")
10
11 # Find hypernyms
12 hypernyms = house_synset.hypernyms()
13 print(f"\nHypernyms of '{house_synset.name()}':")
14 for h in hypernyms:
15     print(f"    - {h.name(): {h.definition()}"")
```

- (b) Using the `path_similarity` function (see the tutorial), write the code to find the path similarity between the first senses of the two words, *mouse* and *horse*. What is the similarity? What is the path similarity between the first senses of *horse* and *vacation*? Which pair is more similar?

```
18 # b
19 # Get the first synset of each word
20 mouse_synset = wn.synsets('mouse')[0]
21 horse_synset = wn.synsets('horse')[0]
22 vacation_synset = wn.synsets('vacation')[0]
23
24 print(f"mouse -> {mouse_synset.name(): {mouse_synset.definition()}"")
25 print(f"horse -> {horse_synset.name(): {horse_synset.definition()}"")
26 print(f"vacation -> {vacation_synset.name(): {vacation_synset.definition()}"")
27
28 # Compute path similarity
29 sim_mouse_horse = mouse_synset.path_similarity(horse_synset)
30 sim_horse_vacation = horse_synset.path_similarity(vacation_synset)
31
32 print(f"\nPath similarity (mouse, horse): {sim_mouse_horse}")
33 print(f"Path similarity (horse, vacation): {sim_horse_vacation}")
34
35 # Compare
36 if sim_mouse_horse > sim_horse_vacation:
37     print("\n=> (mouse, horse) is more similar")
38 else:
39     print("\n=> (horse, vacation) is more similar")
```

The first synsets are:

- `mouse.n.01`: any of numerous small rodents typically resembling diminutive rats
- `horse.n.01`: solid-hoofed herbivorous quadruped domesticated since prehistoric times
- `vacation.n.01`: leisure time away from work devoted to rest or pleasure

The path similarity between the first senses of *mouse* and *horse* is 0.1429, and the path similarity between the first senses of *horse* and *vacation* is 0.0455.

Therefore, *mouse* and *horse* are more similar.

- (c) Download the Glove word embedding vectors (glove.6B.zip) from this website: <https://nlp.stanford.edu/projects/glove/>. We will use glove.6B.50d.txt. Write the code to compute cosine similarity between two words using the Glove word embedding vectors. Don't use any pre-built function/package for cosine similarity. You can use `numpy`.

```
41 # c
42 import numpy as np
43
44 def load_glove(filepath):
45     embeddings = {}
46     with open(filepath, 'r', encoding='utf-8') as f:
47         for line in f:
48             values = line.split()
49             word = values[0]
50             vector = np.array(values[1:], dtype=np.float32)
51             embeddings[word] = vector
52     return embeddings
53
54 def cosine_similarity(v1, v2):
55     # Compute dot product
56     dot_product = np.dot(v1, v2)
57     # Compute magnitudes
58     magnitude = np.linalg.norm(v1) * np.linalg.norm(v2)
59     return dot_product / magnitude
60
61 # Load GloVe vectors
62 glove = load_glove('glove.6B.50d.txt')
63
64 # Compute cosine similarities
65 sim_mouse_horse_glove = cosine_similarity(glove['mouse'], glove['horse'])
66 sim_horse_vacation_glove = cosine_similarity(glove['horse'], glove['vacation'])
67
68 print(f"\nCosine similarity (mouse, horse): {sim_mouse_horse_glove:.4f}")
69 print(f"Cosine similarity (horse, vacation): {sim_horse_vacation_glove:.4f}")
70
71 if sim_mouse_horse_glove > sim_horse_vacation_glove:
72     print("=> (mouse, horse) is more similar")
73 else:
74     print("=> (horse, vacation) is more similar")
```

- (d) What is the cosine similarity between *mouse* and *horse*? What is the cosine similarity between *horse* and *vacation*? Which one is bigger?

The cosine similarity between *mouse* and *horse* is 0.4356, and the cosine similarity between *horse* and *vacation* is 0.3102.

Therefore, *mouse* and *horse* have a bigger cosine similarity.